

Lab 3 — Serial I/O: UART and the Loopback Test

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	2 hrs
Introduces	framing, baud, driver ring buffers, the GPIO matrix
Prerequisites	Labs 0-2.
Beyond the board	One jumper wire (GPIO17 to GPIO18). A logic analyzer is optional but recommended.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§3.1 objective**, **§3.2 parts and wiring**, **§3.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

3.1 Objective & TM4C bridge

Send and receive real bytes on real wires, and prove it with a loopback rather than by trusting a log line. This is also your first use of the logic analyzer on a protocol.

TM4C BRIDGE

This is the bare-metal UART lab. There you enabled the UART clock, computed the baud divisors **IBRD** and **FBRD** by hand from the system clock, packed the line format into **LCRH** (**0x60** for 8N1), routed the pins through **AFSEL** and **PCTL**, then polled the **TXFF** and **RXFE** flags one character at a time. Every one of those steps still happens — the driver just does them. The payoff is that you can now say you have done it both ways, which is exactly the kind of depth that separates you from someone who only knows the API.

3.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	Required	—

Item	Status	Notes
One male-to-male jumper	in the ELEGOO kit	The loopback wire.
Logic analyzer	optional but do it	First real protocol capture.

From	To	Why
GPIO17 (TX)	GPIO18 (RX)	Everything transmitted comes straight back in.

What a loopback is and why it is not a trick. You connect the board's transmit pin to its own receive pin, so the chip talks to itself. That proves your configuration, your framing and your read path all work *without needing a second device to be correct too*. Pull the wire and the received data stops, which is the proof that bytes were genuinely moving on copper rather than being faked in software. This is standard professional bring-up practice, not a shortcut — it is how you isolate "is my UART broken" from "is the other device broken."

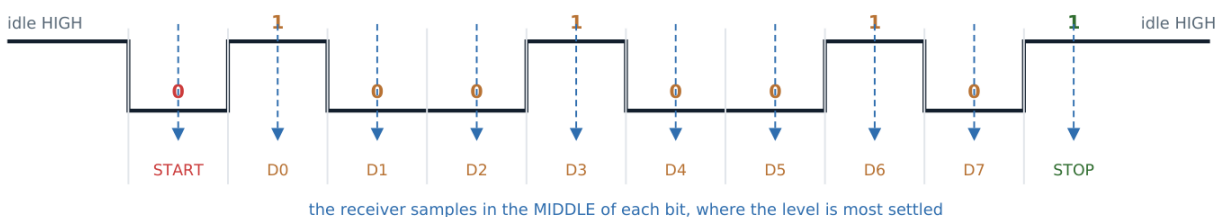
NOTE**Analyzer hookup**

GND to any board GND **first**, then CH0 to GPIO17 and CH1 to GPIO18. Sample at 4 MHz, add the UART decoder, set it to 115200 baud, 8 data bits, no parity, 1 stop bit. Rename the channels TX and RX before adding the decoder. You should see your `ping` bytes decoded as ASCII.

3.3 How it works

One 8N1 character on the wire: 8 data bits, No parity, 1 stop bit

There is no clock wire. Both ends agreed on the bit rate in advance, and the receiver times every bit from the falling START edge.

**Why a START bit exists at all**

The line idles HIGH, so a falling edge is unambiguous: it cannot be mistaken for idle. That edge starts the receiver's timer.

Why a STOP bit exists

It forces the line back HIGH, so the NEXT falling edge is guaranteed to be a real start bit. Without it, two characters could run together indistinguishably.

What goes wrong without a clock wire

If the two ends disagree on the bit rate, the receiver samples at the wrong instants and drifts further off with every bit — which is why the LAST bits of a character corrupt first.

Garbage on the monitor is almost always a baud mismatch, not a wiring fault.

10 bits per byte, so 115200 baud is ~11.5 KB/s.

Figure 3.1. One 8N1 character. With no clock wire, the falling START edge starts the receiver's own timer and it samples in the middle of each bit — which is why a baud mismatch corrupts the last bits of a character first.

One UART character frame (8N1) on the wire.

The line idles HIGH. One character consists of, in order: START bit (0), then eight data bits D0 through D7 least-significant first, then a STOP bit (1) which returns the line to idle.

There is no clock wire. Both ends agreed on the bit rate in advance, and the receiver times every bit from the falling START edge, sampling in the MIDDLE of each bit where the level is most settled.

Why a START bit exists at all: the line idles HIGH, so a falling edge is unambiguous and cannot be mistaken for idle. That edge starts the receiver's timer.

Why a STOP bit exists: it forces the line back HIGH, so the NEXT falling edge is guaranteed to be a real start bit. Without it, two characters could run together indistinguishably.

What goes wrong without a clock wire: if the two ends disagree on the bit rate, the receiver samples at the wrong instants and drifts further off with every bit, which is why the LAST bits of a character corrupt first. Garbage on the monitor is almost always a baud mismatch, not a wiring fault.

10 bits per byte, so 115200 baud is about 11.5 KB/s.

Asynchronous means there is no clock wire

SPI and I²C send a clock alongside the data, so the receiver always knows exactly when to sample. UART does not. There is one wire each way and **no shared clock**, which is what "asynchronous" means and which creates the central problem: how does the receiver know where one bit ends and the next begins?

The answer is that both ends agree in advance on the bit rate — the **baud rate** — and the receiver then times the bits itself. The line idles HIGH. A falling edge marks the **start bit**, and that edge is the receiver's synchronisation event: it starts a timer, waits half a bit period to land in the middle of the start bit, then samples every bit period afterward. That is the whole mechanism, and it explains everything about UART:

- **Why a baud mismatch produces garbage rather than nothing.** The receiver still sees edges and still samples; it just samples at the wrong moments, so it decodes plausible-looking wrong bytes. Garbage on the terminal almost always means a baud mismatch.
- **Why there is a stop bit.** It returns the line to HIGH so the next falling edge is unambiguously a new start bit.
- **Why the frame is short.** The receiver's clock drifts against the sender's, and the error accumulates across the frame. Ten bits is short enough that ordinary clock tolerance does not accumulate into a sampling error.

8N1 is the near-universal frame format and names the three choices: 8 data bits, No parity, 1 stop bit. Parity is a single error-detection bit that is essentially never used on modern short links. This is exactly the `LCRH = 0x60` you wrote on the TM4C, spelled out in words.

What the driver adds

On the TM4C you polled a status flag and moved one character at a time, so if your loop was busy when a byte arrived, that byte was lost. `uart_driver_install` gives you an **interrupt-driven receive ring buffer**: an ISR inside the driver catches every incoming byte the instant it arrives and parks it in a circular buffer, so you can go and do other work and collect a whole batch later. That is a real

functional upgrade, not just convenience, and it is why the read call takes a timeout instead of blocking forever.

3.4 Procedure & code

1. Create the project and set the target. Fit the loopback jumper.
2. Type the listing and flash it.
3. Watch the monitor, then pull the jumper and watch the echoes stop.

```
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "esp_log.h"

#define U_PORT  UART_NUM_1
#define TX_PIN  17
#define RX_PIN  18
#define BUF      1024

void app_main(void) {
    uart_config_t cfg = {
        .baud_rate    = 115200,           // just a number. No IBRD/FBRD by hand.
        .data_bits    = UART_DATA_8_BITS,
        .parity        = UART_PARITY_DISABLE,
        .stop_bits     = UART_STOP_BITS_1,
        .flow_ctrl     = UART_HW_FLOWCTRL_DISABLE,
        .source_clk    = UART_SCLK_DEFAULT,
    };
    uart_driver_install(U_PORT, BUF, 0, 0, NULL, 0); // installs the RX ring buffer + ISR for you
    uart_param_config(U_PORT, &cfg);
    uart_set_pin(U_PORT, TX_PIN,
RX_PIN,           // pins via the GPIO matrix -- not locked to PA0/PA1
                UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);

    const char *msg = "ping\r\n";
    uint8_t rx[BUF];
    while (1) {
        uart_write_bytes(U_PORT, msg, strlen(msg));
        int n = uart_read_bytes(U_PORT, rx, sizeof(rx) - 1, pdMS_TO_TICKS(50));
        if (n > 0) { rx[n] = 0; ESP_LOGI("lab3", "looped back: %s", (char *)rx); }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

3.5 Code walkthrough, line by line

Thirty-four lines. The driver hides the register work done by hand on a bare-metal part, so most of this is configuration — but each field maps onto something you once set yourself.

Line 1 · #include

Standard C, here only for `strlen` on line 29. Worth noticing that a "firmware" program still leans on the ordinary C library.

Line 7 · `#define U_PORT UART_NUM_1`

Selects UART peripheral **1**. The chip has three, and you deliberately avoid UART0 because the console log already uses it — sharing it would tangle your data with the boot messages and monitor output. Choosing a free peripheral instance is a small habit that prevents a confusing class of bug.

Line 10 · `#define BUF 1024`

Serves double duty: the driver's receive ring-buffer size on line 21, and the size of the local `rx` array on line 27. Naming it once means those two can never drift apart.

Line 14 · `.baud_rate = 115200`

The bit rate as a plain number. The driver computes the clock divisors internally. **TM4C bridge:** this single field replaces the `IBRD / FBRD` arithmetic you did by hand — dividing the system clock by $16 \times \text{baud}$, splitting into integer and fractional parts, and writing both registers.

Lines 15-17 · `.data_bits / .parity / .stop_bits`

These three spell out the frame format, conventionally written **8N1**: eight data bits per character, No parity bit, and one stop bit to return the line to idle. Parity is a single extra error-detection bit that is essentially never used on modern short links, and the stop bit exists so the receiver can unambiguously recognise the next falling edge as the start of a new character. **TM4C bridge:** this is your `LCRH = 0x60`, written as named options instead of hand-packed bits. Being able to say "0x60 sets WLEN to 8 bits and enables the FIFO" is exactly the kind of depth that distinguishes you from someone who only knows the API.

Line 18 · `.flow_ctrl = UART_HW_FLOWCTRL_DISABLE`

No RTS/CTS hardware handshaking. Those are two extra wires that let a receiver tell a sender to pause; unnecessary at these rates over a jumper, and standard to disable.

Line 19 · `.source_clk = UART_SCLK_DEFAULT`

Lets the driver pick which clock the baud generator counts from. Some sources survive light sleep and some do not, which is why the choice exists at all.

Line 21 · `uart_driver_install(U_PORT, BUF, 0, 0, NULL, 0);`

Six arguments: the port, the **RX buffer size**, the **TX buffer size** (`0` means transmit synchronously with no buffering), an **event queue depth** and **queue handle** (`0 / NULL` means we do not want the event queue), and **interrupt flags** (`0` for defaults).

The payoff is that this one call stands up an **interrupt-driven receive ring buffer**: a driver ISR catches every incoming byte the instant it arrives and parks it, so you cannot miss one while doing something else. **TM4C bridge:** on the TM4C you polled `RXFIF` and grabbed one character at a time, and a byte arriving while you were busy was simply lost. This is a real functional upgrade, not just convenience.

Line 22 · `uart_param_config(U_PORT, &cfg);`

Applies the format struct. Passed by address so the driver can read it. Note the ordering: install first, then configure — the driver must exist before it can be configured, and swapping these two lines fails.

Lines 23-24 · `uart_set_pin(U_PORT, TX_PIN, RX_PIN, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);`

Routes the signals to pins. Argument order is **TX then RX**, and swapping them is a classic silent failure. The two `UART_PIN_NO_CHANGE` values say "we are not using the RTS and CTS pins," matching line 18.

TM4C bridge: that free choice of pins is the **GPIO matrix** at work. On the TM4C, UART1 was tied to specific pins you enabled through `AFSEL` and selected a function for in `PCTL`. Here almost any GPIO can carry the signal.

Line 26 · `const char *msg = "ping\r\n";`

The test string. `\r\n` is carriage-return plus line-feed, the conventional serial end-of-line — a holdover from teletypes, where CR returned the print head and LF advanced the paper. Many terminals need both.

Line 27 · `uint8_t rx[BUF];`

The receive buffer, declared as **raw bytes** rather than `char`. That is deliberate and honest: what arrives on a wire is bytes, and whether they happen to be text is your interpretation, applied on line 31.

Line 29 · `uart_write_bytes(U_PORT, msg, strlen(msg));`

Transmits. `strlen(msg)` gives the length, and note it counts **6** here — `ping\r\n` — because `strlen` does *not* include the terminating `\0`. That is correct: you want the terminator to stay out of the wire data.

Line 30 · `int n = uart_read_bytes(U_PORT, rx, sizeof(rx) - 1, pdMS_TO_TICKS(50));`

Pulls up to `sizeof(rx) - 1` bytes out of the ring buffer, waiting at most 50 ms, and returns `n`, the count actually received — which may be 0. Two details matter.

The `- 1` reserves one byte so line 31 has room to write a terminator. Ask for the full `sizeof(rx)` and a completely full buffer would leave nowhere to put it, and you would write one past the end.

The **timeout** means this call cannot block forever. Because the driver has been buffering in the background, a 50 ms window is plenty to collect whatever arrived during the previous half-second.

Line 31 · `if (n > 0) { rx[n] = 0; ESP_LOGI("lab3", "looped back: %s", (char *)rx); }`

Three things in one line. The **guard** `n > 0` skips the log when nothing arrived, which is what makes pulling the jumper visibly stop the output.

`rx[n] = 0;` writes the string terminator. C strings must end in a `\0` and the incoming bytes do not include one, so `%s` would otherwise run off into whatever memory follows. Writing at index `n` is exactly right: the received bytes occupy 0 through `n-1`.

`(char *)rx` is a cast. `rx` is `uint8_t*` (raw bytes) but `%s` expects `char*`, so the cast tells the compiler "treat these bytes as characters." Nothing changes in memory — a cast between pointer types only changes how the compiler *interprets* the same address.

Line 32 · `vTaskDelay(pdMS_TO_TICKS(500));`

Paces the loop at 2 Hz and yields the CPU. Slow enough to read in the monitor, fast enough that pulling the jumper produces obvious feedback.

3.6 Docs & references

DOCS

Trace every line to its source

`uart_config_t`, `uart_driver_install`, `uart_param_config`, `uart_set_pin`, `uart_write_bytes`, `uart_read_bytes` — ESP-IDF API Reference → Peripherals → **UART**. Read the "Overview" and "How to Use" sections in full; they lay out this exact install-configure-setpin order and explain why it matters.

Which UART is the console — `menuconfig` → Component config → ESP System Settings → Channel for console output. Confirm for yourself that it is UART0, which is why this lab uses UART1.

GPIO matrix — ESP32-S3 Technical Reference Manual, "IO MUX and GPIO Matrix" chapter, if you want the hardware story behind arbitrary pin routing.

`strlen` — any C reference. Note it does *not* count the terminating zero, which is why the transmitted length is 6 for `"ping\r\n"`.

3.7 Verify, troubleshoot, your turn

Verify. With the jumper fitted, the monitor prints `looped back: ping` twice a second. Pull the jumper and the prints stop because nothing feeds RX. That on/off behaviour is the proof. On the analyzer you should see the ASCII bytes `p i n g` decoded, and you can measure one bit period as about 8.7 μs, which is 1/115200.

Troubleshoot

Symptom	Cause and fix
Nothing loops back	Jumper not seated, or TX/RX swapped in <code>uart_set_pin</code> . The argument order is TX then RX.
Garbage characters	Baud mismatch between the config and the decoder or terminal.
Board reboots on flash	You used pins that conflict with the console or the flash. Stay away from GPIO 26-37.
Analyzer shows flat lines	Missing ground between the analyzer and the board. Always the first thing to check.

Your turn

Build a tiny command parser: accumulate incoming characters into a persistent buffer until you see a `\r` or `\n`, then act on the completed line — for instance `red`, `green`, `off` driving the Lab 1 LED. Two things will bite you and both are worth the scar. First, a single `uart_read_bytes` call may return a *partial* line, so the buffer has to persist across loop iterations rather than being reset each time. Second, watch out for name collisions with macros already defined by the standard library. Log both in `debug_log.md`.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 3.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03 ←	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.